

The Ontology Doctrine

How to build a Palantir-grade ontology as a solo developer — market-agnostic, trading-focused, with tools anyone can install.

DATE **2026-07-22** AUTHOR **Claude (architect/verifier seat)**

SOURCES **DevCon6 talks 00–09 · Cerebras KB · Inspire inventory · repo audit**

STATUS **v1 – for founder decision**

THE VERDICT, UP FRONT

You are closer than you think, and behind where you think. A Palantir ontology is four primitives — object types, properties, links, actions — held together by three disciplines: a single governed system of record, a tool surface *generated from* the schema, and names/descriptions treated as load-bearing agent context. QuantFlow already has the hardest of the three: a sole-writer Kernel with an append-only event log and content-addressed artifacts, gate-enforced. That's the part enterprises take years to accept, and it's running on your machine.

What you don't have is the ontology itself — the charter of ~14 named, described, linked object types — and the generated tool plane that falls out of it. That is not months of engine work. It is roughly **one week of modeling and two weeks of codegen**, on top of a collaboration plane you proved working this week. The months were lost rebuilding engines, not because the goal was too big. Stop building engines. Write the charter, generate the tools, and let the agents you already wired do research over it.

PART I

What a Palantir ontology actually is — stripped of the paywall

Strip away Foundry's branding and the ontology is a small grammar plus a set of disciplines. None of it is secret, and none of it requires their platform. Their own DevCon6 talks describe it plainly.

The four primitives

- **Object types** — the real-world nouns of the business (*not* mirrors of source tables). One canonical type per real entity.
- **Properties** — typed fields, every one described, with pipeline-noise excluded ("no business value" columns stay out).
- **Links** — pre-wired relationships between types. This is the piece that makes agents reliable: *"having it all modeled... lets us get a lot closer to one-shotting these analyses."*
- **Actions** — the governed verbs. All mutation goes through them; nothing writes to the store directly.

The three disciplines that make it "real"

1. **One governed system of record.** A single truth layer, mutated only through actions, with lineage. This is the cultural mountain for enterprises — hundreds of ERPs, department silos. For you it's a solved problem: the Kernel is the sole writer, enforced by falsified qa/ gates.
2. **Tools follow the ontology, not the reverse.** The single most portable idea from DevCon6: Palantir didn't hand-craft twenty MCP tools. They modeled the object/link/action graph correctly, and generic CRUD + action tools fell out of it for free. That's why their agents chained retrospective → plan → publish → dependency-query without hand-holding.

3. **Names and descriptions are agent context.** *"These LLMs were not trained on your enterprise's data"* — the schema is the grounding an agent reasons over. A vague name that used to cost a human a Slack message now degrades agent decisions directly. Misnomer is "one of the worst anti-patterns."

The proof standard

An ontology is *real* — not marketing — the day an agent performs cross-object synthesis through generated tools in one shot. Palantir's proof was: synthesize a retrospective from Slack + tickets + GitHub, draft OKRs, publish them back as linked objects, then immediately query cross-team dependencies. QuantFlow's equivalent proof, defined now so you can't move the goalposts later:

An orchestrator seat is asked: "What did the last Run on Hypothesis X show, which Evaluation gated it, and should we re-run against the newer Dataset?" — and answers correctly using only tools generated from the schema, in one pass, with every step recorded to the Kernel.

– the QuantFlow one-shot test (Phase 4 exit gate)

The honest part about the last few months

The time wasn't lost because the goal was too ambitious. It was lost to **layer confusion** — rebuilding engines (harness, canvas, runtime plumbing) that were already good enough, while the world model those engines exist to serve stayed empty. Your mobile argument yesterday ended at the correct answer: the dock/canvas of heterogeneous agents is the differentiator nothing else ships, and this week the collaboration plane through it became *real* (peer bus + PTY delivery, verified). The missing organ was never another engine. It's the shared world the agents act on. That's the ontology, and it's Part IV of this document.

PART II

What QuantFlow already has — the audited inventory

Mapped against the six-layer agent stack Palantir launched at DevCon6. Status is from verified repo state, not docs.

PALANTIR LAYER	WHAT IT DOES THERE	QUANTFLOW EQUIVALENT	STATUS
Ontology	Substrate representing the organization; how AIP understands agents	SQLite Kernel — sole writer, append-only event log, content-addressed artifacts, schema-generated code	SUBSTRATE HAVE CHARTER GAP
Ontology MCP (OMCP)	One MCP server over the whole graph; CRUD + action tools derived from schema	qf-peer-bus proves the MCP plane works and records to Kernel; but qf_* verbs are hand-grown, not generated	SEED
Agent Engine	The agent primitive; typed sessions, approval gates	Hermes seats (orchestrator / worker / worker2) — BYO harness, the exact pattern Palantir themselves support in SuperRepo	HAVE · PROVEN
Orchestrator	Durable execution, replay, zero-cost suspend	Append-only event log gives replayability; no durable suspend/retry semantics yet (Effect is the candidate)	PARTIAL
Agent SDK / Builder	Pro-code + low-code doors to the same primitives	qf-kernel-schema codegen (schema → typed code) — the OSDK move, already yours	HAVE

PALANTIR LAYER	WHAT IT DOES THERE	QUANTFLOW EQUIVALENT	STATUS
Agent Manager	Fleet telemetry and observability	Canvas + dock (seat spawn rail), agent.log, trajectory artifacts. Palantir <i>retreated</i> from their graph view; your operator wants it — this is the moat	HAVE · DIFFERENTIATOR
AIP Evolve	Automatic model/cost optimization via experiment search	Future: Evaluation history as fitness function (backtest metrics beat LLM-as-judge)	LATER
Workshop / frontend	Form-based apps over the ontology	Electron canvas — spatial, operator-grade, already renders live seats	HAVE
Two-layer permissioning	User-scoped token + category deny-list for external AI clients	Transport-level read/write separation only; category deny-list is a clean future steal for any cloud surface	LATER

Read that table honestly: five of nine layers are HAVE, and one of the HAVEs (the canvas) is something Palantir explicitly walked away from shipping. The two gaps that matter — the charter and the generated tool plane — are the cheapest rows on the board. That is the entire course correction.

Also already banked this week

- **Real agent-to-agent collaboration** — orchestrator → worker peer message injected into a live TUI, auto-processed, replied to via `send_to_peer`, both legs recorded as content-addressed trajectory artifacts. This was the multi-month blocker; it is done and verified.
- **Verification culture** — falsified gates (bait → red → restore → green), cold worktree verification, provenance recomputation. This *is* governance; Part VI just aims it at the schema.

- **Trajectory store** — every peer message is already a distilled, structured artifact: exactly the "distill-then-embed" shape the Cerebras recall layer will want, done early by accident of discipline.

PART III

The replication kit — every Foundry organ, solo-dev edition

The comparison you asked to preserve: Palantir's ideology and tools on the left, the openly available equivalent on the right. Nothing in this table is behind a paywall, a free-tier cap, or a contract. Most of it is already in your repo.

FOUNDRY COMPONENT	PALANTIR'S IDEOLOGY	SOLO-DEV TOOL	STATUS
Ontology object store	One governed truth layer for the organization	SQLite via <code>bun:sqlite / node:sqlite</code> — the Kernel. Local, owned, no rate limits, sub-millisecond	HAVE
Ontology-as-code (<code>ontology.mts</code>)	Objects/links/actions defined in TypeScript; live regen; schema drift = lint error	A TypeScript charter module in your monorepo; <code>qf-kernel-schema</code> codegen regenerates typed code; a lint that fails when agent tool defs drift from schema	PHASE 1
OSDK codegen	Schema change → SDK regenerates → type errors surface in dependent code	<code>qf-kernel-schema</code> already generates code from schema — extend it to emit tool definitions too	HAVE (EXTEND)
Actions / write-back	All mutation through governed action types; agents publish results as linked objects	Kernel commands via <code>execute()</code> over the append-only event log — stricter than Foundry's, and already gate-enforced	HAVE

FOUNDRY COMPONENT	PALANTIR'S IDEOLOGY	SOLO-DEV TOOL	STATUS
OMCP server	One MCP server over the modeled graph; tools derived, not hand-written; client-agnostic	@modelcontextprotocol/sdk — the exact stack qf-peer-bus already ships on. Generate per-type read tools + per-action write tools from the charter	PHASE 2
Pipelines	System-of-record data flows in via pipelines, never via manual actions (anti-Golden-Hammer)	Bun scripts + cron, writing through Kernel commands with an ingest trace. One script per source	PHASE 3
Workshop / OSDK apps	Human surfaces over the ontology	The Electron canvas + dock — already superior for your operator (they retreated from the graph; you ship it)	HAVE
Agent Engine + harnesses	BYO third-party harness supported today; Agent SDK "coming soon"	Hermes seats + any MCP-speaking agent (Claude Code, Codex, anything). Model-agnostic by construction	HAVE · PROVEN
Orchestrator (durable execution)	Durable ledger, replay with idempotency keys, zero-cost suspend	Event-log replay now; Effect (typed retries/errors, from your Inspire inventory) when Runs get long-horizon	PHASE 4+
Agent Manager telemetry	Zero-config fleet observability; time attribution before optimization	agent.log + trajectory artifacts + canvas. Add span timing when the loop is live — measure before optimizing	PARTIAL
Recall / knowledge layer	(Cerebras pattern, same doctrine) distill-then-embed, hybrid retrieval, RRF fusion	SQLite FTS5 + sqlite-vec, RRF k=60, age decay. Trajectories are already distilled — half done	PHASE 5
Two-layer permissioning	User-scoped token + marking deny-list so	Build only when an external/cloud agent first	PHASE 5

FOUNDRY COMPONENT	PALANTIR'S IDEOLOGY	SOLO-DEV TOOL	STATUS
	lower-trust AI clients can't exfiltrate categories	touches the surface: a category deny-list in the generated MCP server	
AIP Evolve	Bounded experiment search over models/configs	Evaluation history as fitness. Sharpe/drawdown/hit-rate are objective — better than LLM- as-judge, and unique to your domain	PHASE 6
Multi-tenant ACLs, marketplace, fleet scale	Enterprise machinery	Not your problem. A solo operator replicating these is cosplay — skip deliberately	SKIP

The 50-object cap, revisited

The free-tier limit that helped push you off Foundry is worth one more look — because Palantir's own governance talk implies a well-modeled ontology is *small*. God Objects and type-per-system silos are what bloat counts. The charter in Part IV needs **~14 object types**. The cap was never your real constraint; **ownership and real-time were** — and both are solved by staying local. Use the free tier, if at all, as a modeling lab to sanity-check the charter. Never as a dependency. (This is also your own research verdict from talk 00: *borrow the doctrine, don't build on the platform* — availability fails decisively; there is nothing to `npm install`.)

PART IV

The QuantFlow ontology charter — v1 proposal

Domain-driven design, in Palantir's own ordering: understand the domain → design the ontology → map source data onto it. Never the reverse. The domain is **quantitative research** — not football, not HyperLiquid. Markets plug in as data; the

research loop is invariant. That is how you stay market-agnostic without being empty.

Two planes, one agent layer

- **RESEARCH PLANE** — the invariant loop. Identical whether the instrument is a football spread, a perp contract, or an equity. This is the spine.
- **MARKET PLANE** — pluggable per domain. Fed by pipelines only. Adding a new market adds *rows*, not object types.
- **AGENT PLANE** — who does the work. Already largely real in the Kernel (species, sessions, trajectories).

flowchart LR

```
subgraph RP["RESEARCH PLANE (invariant)"]
```

```
  H["Hypothesis"] -->|TESTS_ON| D["Dataset"]
```

```
  H -->|TESTED_BY| R["Run"]
```

```
  R -->|PRODUCES| A["Artifact"]
```

```
  A -->|EVALUATED_BY| E["Evaluation"]
```

```
  E -->|GATES| REP["Report"]
```

```
end
```

```
subgraph AP["AGENT PLANE"]
```

```
  AD["AgentDefinition"] -->|SPAWNS| AS["AgentSession"]
```

```
  AS -->|DRIVES| R
```

```
  AS -->|EMITS| T["Trajectory (Artifact)"]
```

```
end
```

```
subgraph MP["MARKET PLANE (pluggable)"]
```

```
  V["Venue"] -->|LISTS| I["Instrument"]
```

```
  I -->|QUOTED_BY| Q["Quote"]
```

```
  I -->|HAS| ME["MarketEvent"]
```

```
end
```

```
H -->|TARGETS| I
```

```
D -->|SNAPSHOTS| Q
```

The object types (~14, each with a mandatory description)

PLANE	OBJECT TYPE	WHAT IT IS (THE DESCRIPTION AGENTS WILL READ)	KEY LINKS
R	Hypothesis	A falsifiable claim about market behavior, with success criteria declared before any Run	TARGETS In TESTS_ON D
R	Dataset	A point-in-time-fenced snapshot of market data a Run may read. Immutable once fenced	SNAPSHOTS Quote/Mark
R	Run	One canonical type. An execution of a Hypothesis against a Dataset. Backtest vs screen vs sim is a property, never a subtype	TESTED_BY← · PRODUCES · DRIVEN_B AgentSessio
R	Artifact	Content-addressed output — trajectory, result table, chart, report body. Already live in the Kernel	PRODUCED_B EVALUATED_ Evaluation
R	Evaluation	A Critic's scored judgment of an Artifact against the Hypothesis's declared criteria. Gates publication	GATES Repo
R	Report	The publishable synthesis. Cannot exist without a passing Evaluation — enforced, not suggested	DERIVED_FR Artifact
R	Mission	Workspace-level intent: the standing questions this desk is working. Scopes search and recall	CONTAINS H
A	AgentDefinition	A species: harness, model, profile, tool grants. Already Kernel rows	SPAWNS Age
A	AgentSession	A live seat on the canvas. Owns trajectories it emits	DRIVES Run Artifact
M	Venue	Where instruments trade or are quoted: a sportsbook, an exchange	LISTS Inst
M	Instrument	A tradeable/betable thing: a game line, a perp, a ticker. The market-agnostic pivot point	QUOTED_BY HAS Market

PLANE	OBJECT TYPE	WHAT IT IS (THE DESCRIPTION AGENTS WILL READ)	KEY LINKS
M	Quote	A priced observation at a timestamp: odds, bid/ask, close. Pipeline-fed only	OF Instrument
M	MarketEvent	A discrete occurrence affecting an instrument: kickoff, injury news, funding event, settlement	ON Instrument
M	Position	(Later — only when real capital exists.) An actual stake taken on an Instrument per a Report	JUSTIFIED_

Actions (the governed verbs — deliberately few)

Coherent verbs, not property-level sprawl. Anything pipeline-shaped (quotes, events) gets *no* action at all — that's the Golden Hammer rule.

- propose_hypothesis · fence_dataset · start_run / complete_run · publish_artifact (exists) · record_evaluation · publish_report (rejects without passing Evaluation) · register_instrument

What the charter looks like as code (Phase 1's actual deliverable)

```
// ontology/research.ts – the charter is a module, not a document.
// Every type and property carries a description: agents read these.
defineObjectType({
  apiName: "Run",
  description: "One execution of a Hypothesis against a fenced Dataset. " +
    "Kind (backtest|screen|simulation) is a property – never a subtype.",
  status: "experimental",          // lifecycle: experimental → active
  properties: {
    kind:      { type: "enum", values: ["backtest","screen","simulation"],
                  description: "Execution mode; differentiates by composition, not typ
    startedAt: { type: "timestamp", description: "Wall-clock start" },
    status:    { type: "enum", values: ["running","complete","failed"],
                  description: "Reported by the execution environment, never set by ha
  },
  links: {
    hypothesis: { to: "Hypothesis", kind: "TESTED_BY", description: "The claim under
    dataset:    { to: "Dataset",    kind: "READS",      description: "Fenced data sna
    artifacts:  { to: "Artifact",   kind: "PRODUCES",   many: true },
```

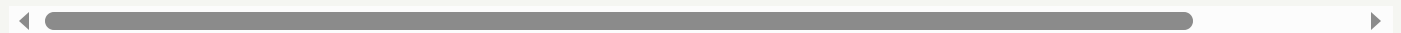
```
    session:    { to: "AgentSession", kind: "DRIVEN_BY", description: "The seat that
},
actions: {
  start_run:    { description: "Begin executing a Hypothesis against a Dataset" },
  complete_run: { description: "Record terminal status + final artifacts" },
},
});
// From this one module, codegen emits: SQL DDL → Kernel · typed TS client →
// seats · MCP read tools (get/search/traverse) + action tools → the qf server.
// Change the schema, everything regenerates, drift becomes a type error.
```

That last comment is Palantir's SuperRepo demo — the local embedded ontology, the live-regenerating OSDK, schema-drift-as-lint-error — reproduced with your existing codegen and a Bun monorepo. Their version needs a Foundry contract. Yours needs a week.

PART V

The roadmap — six phases, each with a falsifiable gate

Same verification culture you already run: no phase is "done" by prose. Each has an exit gate that can go red. Timeboxes assume you + agent builders at current cadence.



0

DONE

The substrate BANKED

Sole-writer Kernel + event log + content-addressed artifacts. Peer bus with Kernel-recorded trajectories. PTY delivery — live, verified agent collaboration in native TUIs. Canvas + dock with seat spawning. Falsified gate culture. **Do not rebuild any of this. It is finished.**

1

WEEK 1

The charter

Write ontology/ as code: the ~14 types of Part IV, every type and property described, links and actions declared, lifecycle flag on every type. This is a

modeling week, not an engineering month — most shapes already exist informally in the Kernel; you are naming and governing them.

EXIT GATE

Schema lint goes red on: a missing description, a subtype-of-Run clone, a property removed from a non-experimental type. Falsified with a bait commit before it counts.

2

WEEKS 2–3

The generated tool plane

Extend qf-kernel-schema codegen to emit the MCP server: per-type read tools (get / search / traverse-links) and per-action write tools, from the charter, on the @modelcontextprotocol/sdk stack qf-peer-bus already proved. Retire hand-grown qf_* verbs as generated equivalents land. The peer bus stays as the agent ↔ agent plane; this is the agent ↔ world plane beside it.

EXIT GATE

Add a brand-new object type to the charter → its tools appear with zero hand-written tool code, and a Hermes seat lists and calls them cold.

3

WEEK 4

The first market plane

One pipeline, one source, one market — odds or HyperLiquid, your call, the ontology doesn't care. A Bun script on cron writing Instrument / Quote / MarketEvent through Kernel commands with an ingest trace. No actions for pipeline-shaped data (Golden Hammer rule). This is the moment the canvas stops being an empty engine.

EXIT GATE

Every market row traces to an ingest event (provenance recomputable), and a seat answers a cross-object question about real data — "which instruments moved most since yesterday's fence?" — through generated tools only.

4

WEEKS 5–8

The defining loop, agent-run

Orchestrator + workers execute the full loop over the peer bus: Hypothesis → Dataset → Run → Artifact → Evaluation → Report, every step a Kernel action, every conversation a trajectory artifact. Evaluation gates Report publication mechanically (Ragas-style critic scoring against declared criteria). Add Effect-typed retries where Runs get long.

EXIT GATE

The one-shot test from Part I: an orchestrator seat answers "what did the last Run on Hypothesis X show, which Evaluation gated it, should we re-run on the newer Dataset?" correctly, in one pass, tools-only, fully recorded. **This is the day QuantFlow is a real ontology.**

5

MONTHS 2–3

Recall + trust boundaries

The Cerebras layer: FTS5 + sqlite-vec, hybrid retrieval fused with RRF (k=60), age decay (betting knowledge expires), Mission-scoped search. Trajectories are already the distilled shape — never embed raw transcripts. Rule stays absolute: *retrieval never becomes truth without a Kernel command*. First external/cloud agent to touch the surface triggers the marking-style category deny-list in the generated server.

EXIT GATE

A seat's answer cites retrieved Reports/Evaluations with artifact hashes; a deny-listed category provably never crosses the MCP boundary (bait-tested).

6

LATER

Evolve-shaped optimization

Evaluation history becomes a fitness function: which strategies, models, and agent configs actually produce passing Evaluations (which_strategy_knows routing). Backtest metrics are an objective fitness signal LLM-as-judge shops would kill for — your domain hands it to you. Also the finetuning substrate: the trajectory store is training data for the next generation of your own agents.

EXIT GATE

Deferred until Phase 4 has months of Evaluation history. Measuring before optimizing is the whole lesson of their telemetry talk.

PART VI

Governance of one — anti-patterns as lint rules

You have no review board, so governance lives in qa/ gates — the same falsified-gate culture you already run, aimed at the schema. Palantir's six named anti-patterns, translated:

PALANTIR ANTI-PATTERN	HOW IT WOULD APPEAR IN QUANTFLOW	THE GATE THAT BLOCKS IT
God Object	Run absorbing every tool's raw log fields	Property-count ceiling + "no ETL noise" review on charter diffs
Kitchen Sink	Mapping every scraper column 1:1 into Quote	Charter properties require a description that states business meaning — no meaning, no property
Silos	BacktestRun / ScreenerRun / SimRun as separate types; a type per sportsbook	Lint: no type name may embed another type's kind enum; new Venue = row, not type
Action Sprawl	update_run_status + update_run_cost + update_run_timing	One coherent complete_run; action count reviewed per type
Golden Hammer	A write-action for quotes that a pipeline should feed	Market-plane types declare pipelineFed: true → codegen emits no write tools for them

PALANTIR ANTI-PATTERN	HOW IT WOULD APPEAR IN QUANTFLOW	THE GATE THAT BLOCKS IT
Misnomer	"Item", "Data", "Thing" — names agents can't reason over	Non-empty description lint on every type and property (already Phase 1's gate)

- **Extend, don't mutate:** status: "experimental" | "active" on every type. A diff that removes/retypes a property on an *active* type fails CI; capability is added via new linked types. Palantir showed no migration tool either — your schema-diff gate + changelog requirement fills a gap they left open.
- **Descriptions are enforced, not encouraged** — because your own agents read this schema exactly the way AIP's do.

PART VII

The claims ladder — what you can say, and when

You said it yourself: pivoting the marketing to "ontology" means backing it up. The repo-docs lesson applies to marketing too — never claim ahead of what a cold verifier could confirm. Here is the ladder; each rung unlocks on a shipped, gated phase.

TODAY

"An agent-collaboration platform with a governed system of record."
Every agent message is a typed, content-addressed artifact in an append-only log; heterogeneous agents (any model, any harness) collaborate in native TUIs on one canvas. All true, all verified this week.

AFTER P1–P2

"An ontology-defined platform: the agents' tool surface is generated from the schema." This is the load-bearing Palantir claim — tools follow the ontology. Once codegen ships, you can say it without flinching.

AFTER P3–P4

"AI agents doing end-to-end quantitative research over a governed ontology." The OMCP-demo equivalent, with the one-shot proof recorded

in the Kernel as evidence. This is the demo video. This is the pitch.

NEVER

Palantir affiliation or comparison-by-name in marketing · scale/fleet claims · "enterprise permissioning" before Phase 5 ships · any claim a bait test hasn't survived.

The course correction, in three sentences

1. Stop building engines — the substrate, the canvas, and the collaboration plane are done and verified; every additional engine week is the gutter. **2.** Spend one week writing the charter and two generating the tool plane — that's the entire distance between "QuantFlow" and "QuantFlow, an ontology." **3.** Then run the defining loop on one real market and let the recorded one-shot proof be your marketing, because a Palantir-grade ontology isn't a feature list — it's the moment an agent does real cross-object work through tools that fell out of your schema.

APPENDIX

Sources

- DevCon6 00 – The Integration Question · the verdict: borrow doctrine, don't build on platform; availability fails decisively
- DevCon6 05 – SuperRepo & Agent Development · local embedded ontology, ontology-as-code, schema-drift-as-lint, parallel worktrees
- DevCon6 08 – Ontology MCP · tools follow the ontology; one server over the graph; two-layer permissioning; one-shot cross-object synthesis
- DevCon6 09 – Ontology Governance · DDD ordering; the six anti-patterns; extend-don't-mutate; descriptions as agent grounding
- DevCon6 02/03/04/06/07 · durable ledger & suspend; typed context items; time attribution; write-back loop; the graph-view retreat
- Cerebras KB notes · record vs recall split; distill-then-embed; hybrid retrieval + RRF k=60; retrieval never becomes truth without a Kernel command

- INSPIRE-USECASES.md · the defining loop; the three-things-first list; Effect, Ragas, WrenAI candidates
- Repo audit (this week) · peer bus + PTY delivery verified; gate falsification records; trajectory artifacts live

QUANTFLOW · THE ONTOLOGY DOCTRINE · v1 · 2026-07-22

Prepared by the architect/verifier seat. Every phase gate in this document is falsifiable by construction – if a gate can't go red, it isn't a gate.